Passkeys & Presence



Nothing to Phish

[PROJECT REPORT · CC6-IC04]

Secure

Phishing-proof, passwordless sign-in with passkeys, plus a face + hand-gesture liveness check for risky moments.

NEXT-GENERATION PASSWORDLESS AUTHENTICATION

HACKATHON BUILD · OCTOBER 2026

0 PASSWORDS · 0 OTPS · 4/4 ATTACKS BLOCKED

[AT A GLANCE]

Passkeys log you in. Your face and hand prove it's really you when it matters.

Secure is a working sign-in system with no passwords and no SMS codes. Each account is protected by a **passkey**: a key pair that lives on the user's phone, laptop or security key and only works on the real website. When a sign-in looks unusual, or when someone tries to recover an account, the user also shows their **face plus a hand gesture** to the camera. That check is extra protection on top of the passkey and can never log anyone in by itself.

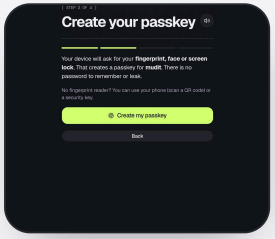
0
PASSWORDS OR OTPS ANYWHERE

4/4
PHISHING ATTACKS BLOCKED LIVE

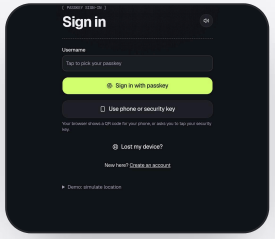
241
AUTOMATED TESTS PASSING

2
PROCESSES, 1 SQLITE FILE. NO OTHER INFRA

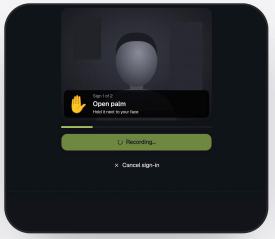
HOW IT WORKS IN FOUR MOMENTS



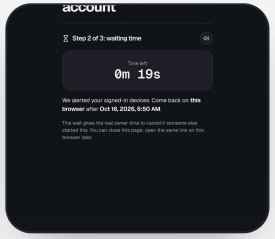
1 **Sign up**: the device creates a passkey. No password is ever set.



2 **Sign in**: one tap with fingerprint, face or PIN. A risk check runs.



3 **Looks unusual?** Show your face + a random sign + your own gesture.



4 **Lost everything?** Recovery code + face check + a waiting period you can cancel.

CONTENTS

01	Problem, goals & requirement coverage	03
02	Big idea and system architecture	05
03	Sign-up and passkey sign-in	10
04	Risk engine and face + gesture liveness	12
05	Multiple devices: add by QR, revoke	17
06	Account recovery	19
07	Phishing-resistance demo	21
08	Admin view and inclusivity	23
09	Threat model, privacy, quality	25
10	Limits, roadmap and demo script	28

9.9k
LINES TS WEB

2.4k
LINES PYTHON BIO

34
API ROUTES

15
SCREENS (PAGES)

All screenshots in this report come from the real app, captured end-to-end by an automated browser with a virtual passkey authenticator. Camera frames use a stand-in silhouette feed.

[01 · THE PROBLEM]

Shared secrets are the weak link

Passwords and one-time codes are secrets that a person can type into the wrong place or read out to the wrong caller. Passkeys remove the secret the user can hand over.

TODAY	WITH SECURE
Passwords Reused across sites, guessed, phished and dumped in breaches. One leak opens many doors. CREDENTIAL STUFFING · BREACH DUMPS	**Passkeys (FIDO2 / WebAuthn)** The private key never leaves the device. The server stores only a public key, so a database leak gives nothing to reuse. NOTHING TO REUSE OR LEAK
SMS one-time codes Intercepted through SIM swaps, or read out to a fraudster pretending to be the bank. SIM SWAP · SOCIAL ENGINEERING	**No OTPs at all** Unusual sign-ins get a face + own-gesture check instead, and only after a valid passkey. There's no code to read out. NOTHING TO READ OUT ON A CALL
Look-alike phishing pages A pixel-perfect clone collects whatever the user types, then replays it on the real site in real time. REAL-TIME RELAY · CREDENTIAL HARVEST	**Origin-bound signatures** The browser signs the site's real origin into every assertion. A clone can't get a valid one. Shown live on p. 21. PHISHING PAGE GETS NOTHING USABLE
Users left out Basic phones, shared devices, limited literacy, and recovery that falls back to "reset by SMS". EXCLUSION · WEAK RECOVERY	**Inclusive by design** Phone-as-key over QR, NFC cards and security keys, a voice button on every screen, and recovery with no SMS fallback. NO TYPING NEEDED TO SIGN IN

[01 · REQUIREMENT COVERAGE]

Every requirement, built and demoable

The problem statement lists seven requirements and three deliverables. Each row below maps one to what was built and where it appears in this report.

#	REQUIREMENT	HOW SECURE MEETS IT	STATUS	PAGE
R1	**Passwordless login**	Discoverable passkeys with `userVerification: "required"`. Platform authenticators, phone-as-key (hybrid QR) and USB/NFC security keys all work.	✓ BUILT	10-11
R2	**Phishing resistance**	Origin, rpID, challenge and signCount are verified on every ceremony. Four attacks from a look-alike clone all fail. Blocked attempts appear in the admin feed.	✓ BUILT	21-22
R3	**Secure account recovery**	Recovery code (100-bit, argon2id) + face/gesture liveness + cooldown + alerts to every signed-in device + cancel + browser binding. No password or SMS fallback.	✓ BUILT	19-20
R4	**Multi-device & revoke**	Add a device by scanning a QR code with an already signed-in phone (6-digit match + passkey approval). Revoke lost devices, which also kills their sessions. Add a security key.	✓ BUILT	17-18
R5	**Inclusivity**	Phone-as-key QR, NFC card / security key, read-aloud button on every screen, big icon + text buttons, 360 px layouts, no typing to sign in.	✓ BUILT	24
R6	**Risk-based checks**	A pure scoring function runs after a valid passkey: new country +40, new browser/OS +20, ≥3 recent failures +20. The result is allow, step-up (liveness) or block.	✓ BUILT	12
R7	**Admin view**	Users, devices (active + removed with reason), an append-only event feed with risk signals, revoked credentials, recovery requests. Admin writes need a fresh passkey.	✓ BUILT	23
D1	**Demo app, end to end**	Sign-up, login, new-device registration and recovery, all captured from the running app in this report.	DELIVERED	10-20
D2	**Phishing demonstration**	The `phish/` clone on a different origin, plus a forged-origin relay that uses a stolen private key and is still rejected.	DELIVERED	21-22
D3	**Short threat model**	Covered, partly covered and not covered threats, in the app at `/threat-model` and in this report.	DELIVERED	25

[02 · THE BIG IDEA]

Passkey first. Presence second.

Most face-login products treat the face as the key. We don't, for three reasons:

1 · Video has no origin

A phishing page can relay the real site's face prompt and forward the victim's camera. A passkey signature is bound to the real site.

2 · Cameras can be faked

Virtual cameras feed recorded or deepfaked video, and the server can't prove the camera is real.

3 · Faces aren't secret

Anyone can find them on social media.

> **Hard rule:** face and gesture verification **never** grants a session on its own. It only clears a step-up on a session that already has a valid passkey, or moves forward a recovery that also needs the recovery code and a cooldown.



[02 · SYSTEM ARCHITECTURE]

Two processes, one SQLite file

The web app owns all UI, API routes and the database. The bio service is stateless and turns camera frames into a pass/fail decision. It is the only holder of the template encryption key.



ATTACKER · phish/ 127.0.0.1:3001

Look-alike sign-in page + proxy server

Relays real challenges, tries to get a passkey, forges headers, relays the face check. Every path ends in SecurityError, ORIGIN_MISMATCH or 401 (pp. 21–22).

× ORIGIN_MISMATCH
signed origin ≠ ORIGIN env

× SecurityError in browser
rpId doesn't match the clone's origin

[02 · STACK & TRUST BOUNDARIES]

Boring tech, sharp edges

Standard libraries do the cryptography. Our own code is mostly glue plus two pure functions (risk and recovery) that are easy to test.

WEB

Next.js 16

App Router, Node runtime, TypeScript strict, Tailwind + shadcn on Base UI.

PASSKEYS

SimpleWebAuthn

Server + browser packages. Attestation none, resident key + UV required.

VALIDATION

zod

Every route parses its input. Errors use a single `{error:{code}}` shape.

DATABASE

SQLite + Drizzle

better-sqlite3. Migrations apply automatically on first use. Challenges live here too, so there's no Redis.

HASHING

argon2id

Recovery codes via `@node-rs/argon2`. Session tokens stored as sha256.

QR & VOICE

qrcode · speech

The QR opens in the phone's own camera app. Browser `speechSynthesis` reads screens aloud.

BIO

FastAPI · uv

Python 3.12, stateless, internal-token auth, in-memory multipart parser.

FACE

InsightFace buffalo_l

SCRFD + ArcFace run directly on onnxruntime CPU, with no Cython build.

HANDS

MediaPipe 0.10

GestureRecognizer: 7 canned classes + 21 3-D landmarks per hand.

DELIBERATELY NOT USED

AUTH SAAS · NEXTAUTH · REDIS · POSTGRES · DOCKER · QUEUES · EMAIL · SMS GATEWAY

WHAT EACH COMPONENT CAN SEE

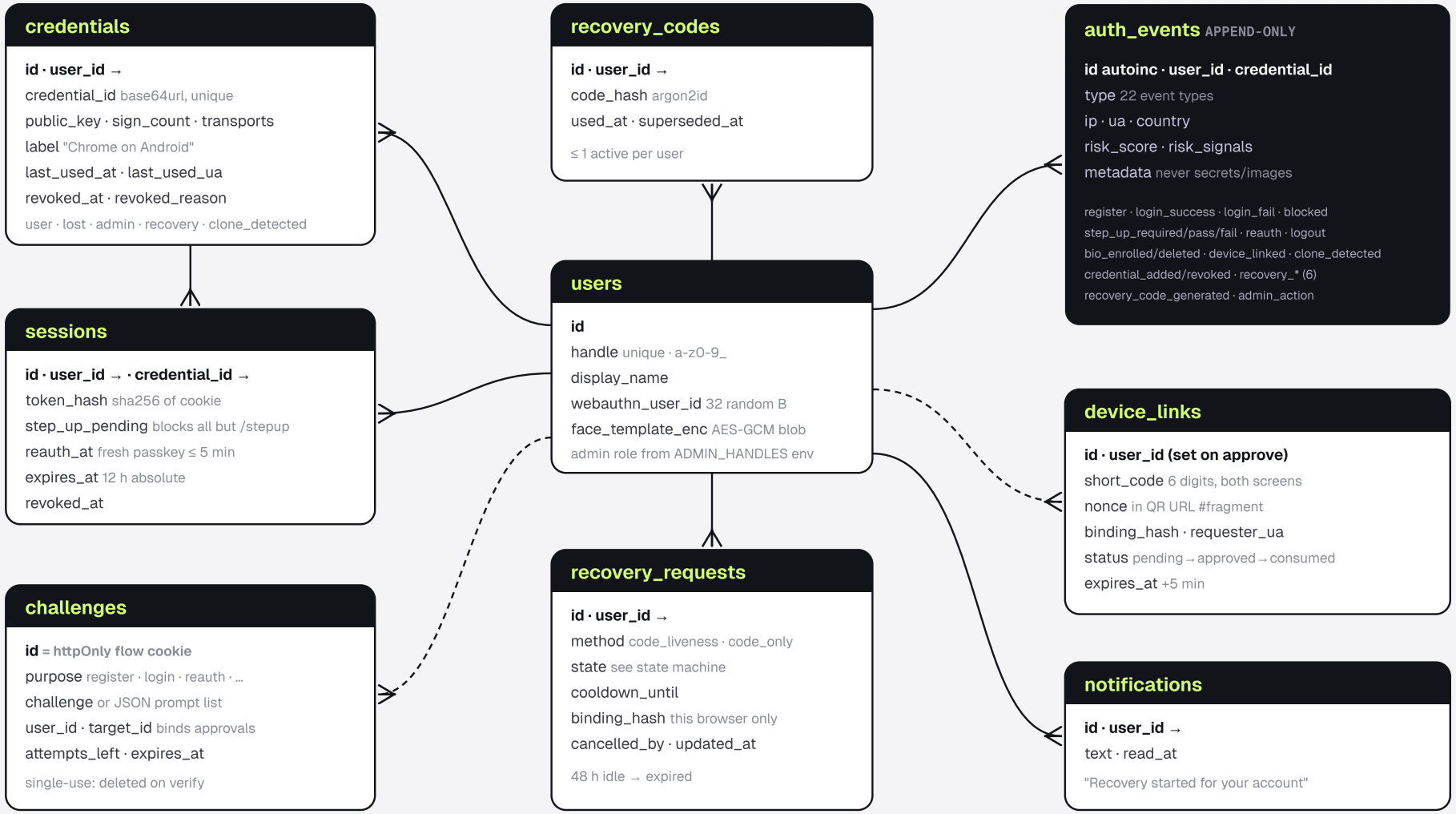
DATA	BROWSER	WEB	DB	BIO
Passkey private key	🔒 authn.	—	—	—
Passkey public key	—	✓	✓	—
Session token	HttpOnly	✓	sha256	—
Recovery code	shown once	in memory	argon2id	—
Camera frames	✓	pass-through	never	in memory
Face + gesture template	—	blob	AES-GCM	✓ decrypts
Template key	—	—	—	✓ only
Match scores	never	✓ → log	event meta	✓

A leaked database exposes **public keys, argon2id hashes and encrypted blobs**. None of these can sign in, recover an account or rebuild a face.

[02 · DATA MODEL]

Nine tables, one append-only log

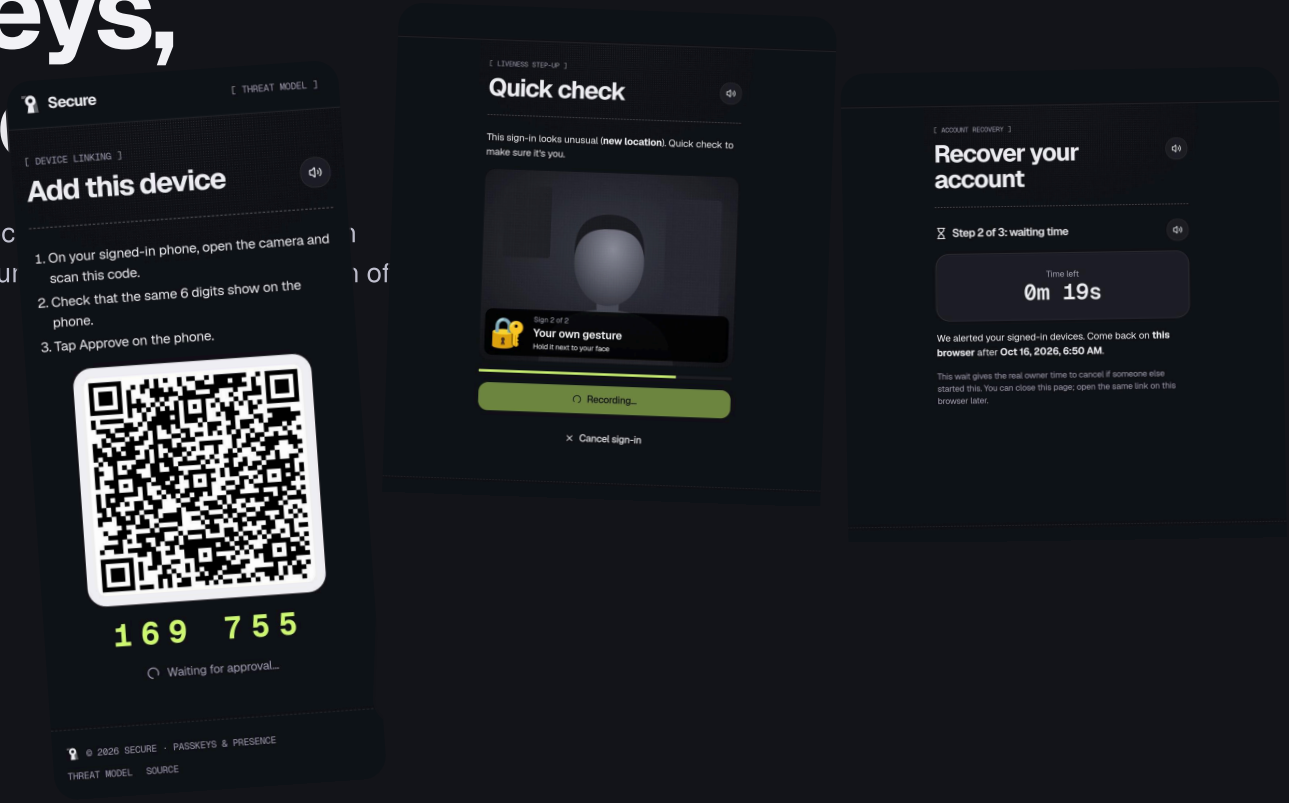
All timestamps are integer ms and all IDs are random UUIDs. Challenges for every ceremony share one table and are deleted on use. `auth_events` rows are never updated or deleted.



03 → 08

The user journeys, screen by scre

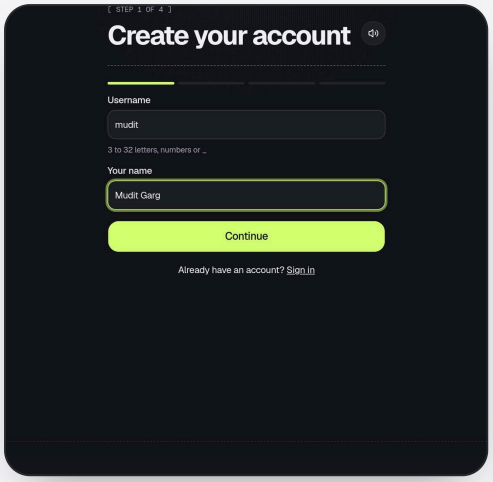
Sign up, sign in, a risky sign-in, adding a phone, losing it, rec … attacker trying every trick along the way. All of it was captu … of the real app.



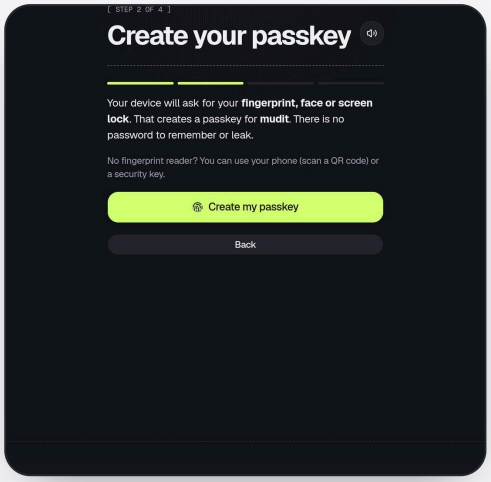
[03 · SIGN-UP]

Four steps, zero passwords

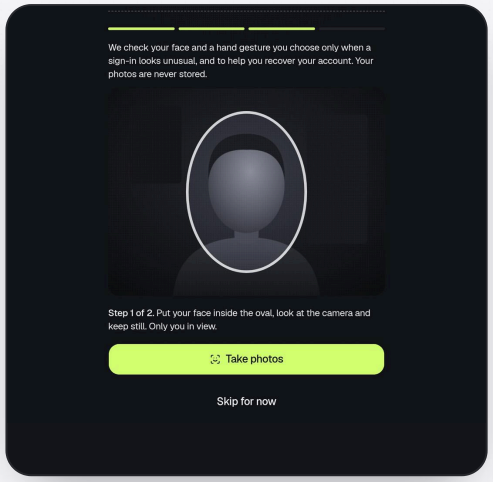
The user picks a handle, the device creates a passkey, face + own gesture are enrolled (this step can be skipped), and a recovery code is shown once.



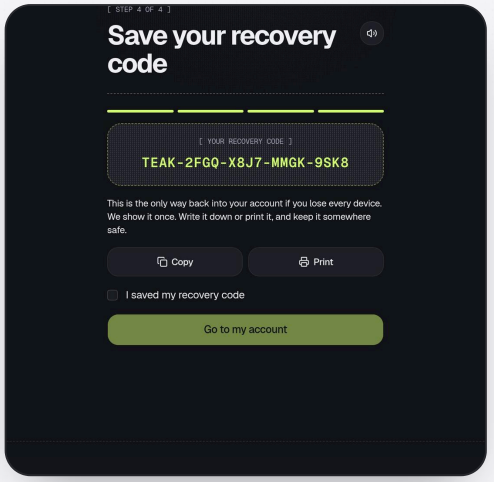
1 **Details**: username + display name. The user row is created only after the passkey verifies.



2 **Passkey**: the OS prompt for fingerprint, face or screen lock. A phone or security key also works.

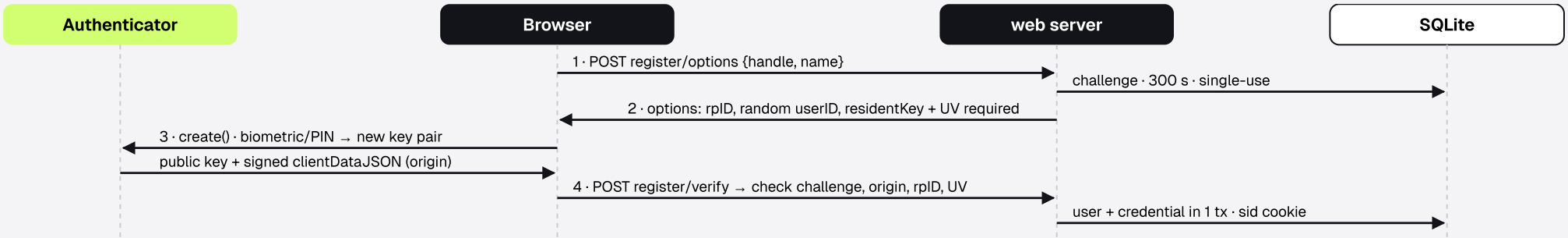


3 **Face + own gesture**: consent first, then 5 face photos + 3 × 5 frames of a hand shape the user picks.



4 **Recovery code**: 100 random bits, Crockford base32, shown once. Stored only as argon2id.

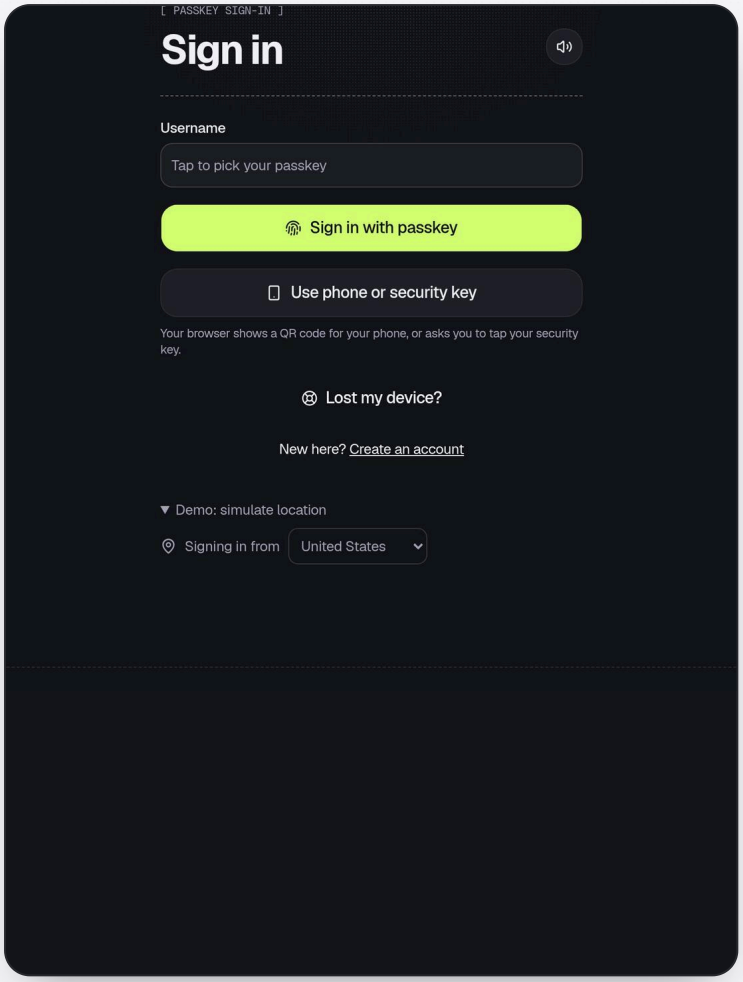
REGISTRATION CEREMONY



[03 · PASSKEY SIGN-IN]

One tap. Six checks behind it.

Discoverable credentials mean no username is needed. The browser autofills passkeys in the username field, or the user taps one button.



/signin: passkey, phone or security key, "Lost my device?". The demo-only *Simulate location* toggle feeds the risk engine.

1. **Load challenge by flow cookie**
   check purpose + expiry, then delete it (single-use)
   → `CHALLENGE_EXPIRED`
2. **Verify signature**
   expected challenge · origin · rpID · UV required
   → `ORIGIN_MISMATCH` + login_fail reason=origin_mismatch
3. **Credential exists, not revoked**
   lost / removed devices can't come back
   → `CREDENTIAL_REVOKED`
4. **signCount must go up**
   regression = cloned authenticator
   → `BLOCKED` auto-revoke as clone_detected
5. **Update counters, reauth_at = now**
   a fresh passkey unlocks sensitive actions for 5 min
6. **Write auth_events row**
   ip · ua · country · risk signals

→ Risk engine decides (next page)

💡 While building this report, the automation re-used a stale copy of a virtual passkey. Check 4 caught the cloned key and **revoked it automatically**.

[04 · RISK-BASED CHECKS]

Ask for more only when it looks unusual

A pure function scores every valid passkey login before a session is issued. Weights and thresholds live in `config.ts`.

SIGNALS

🌍 New country	+40
💻 New browser/OS	+20
🔁 ≥3 fails / 15 min	+20

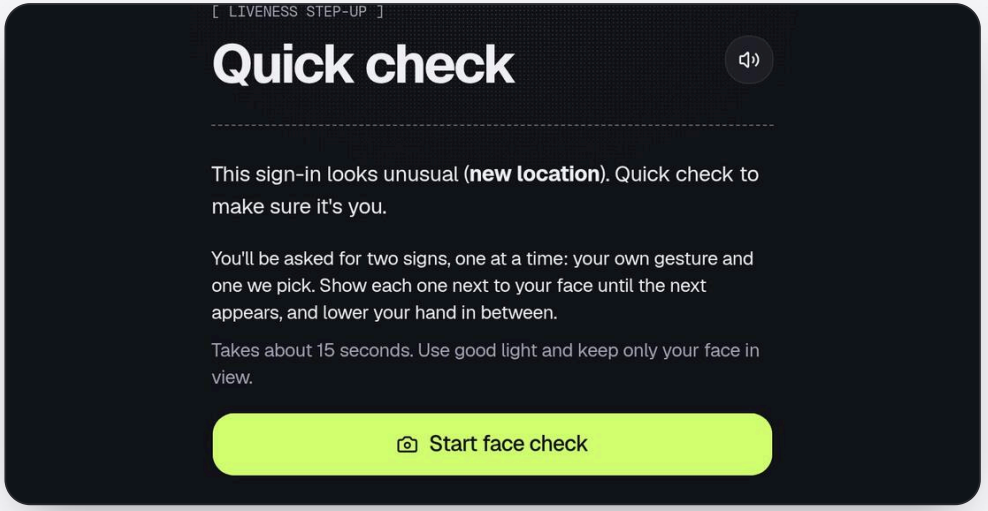
The first login after sign-up scores 0, because there's no history to compare against.

DECISION

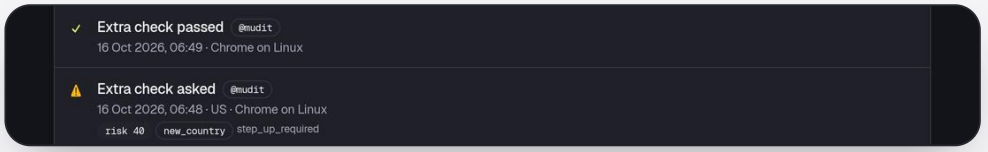


On a **blocked** login, the credential's last-used browser is put back to its old value, so an attacker's browser never becomes the "known" one.

SAME PASSKEY, NEW COUNTRY → STEP-UP



Only the coarse reason is shown ("new location"), never the score.

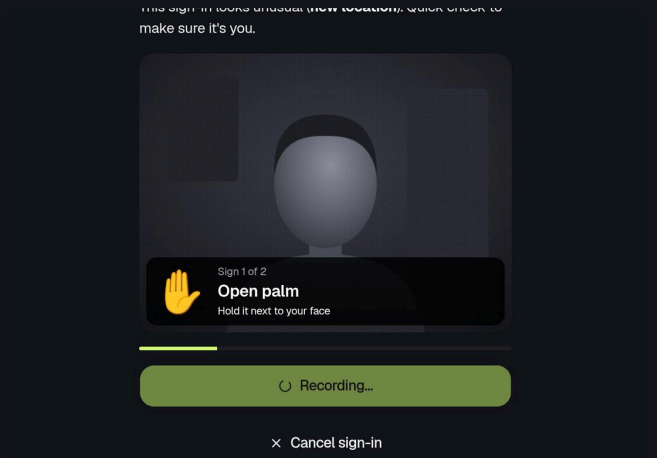


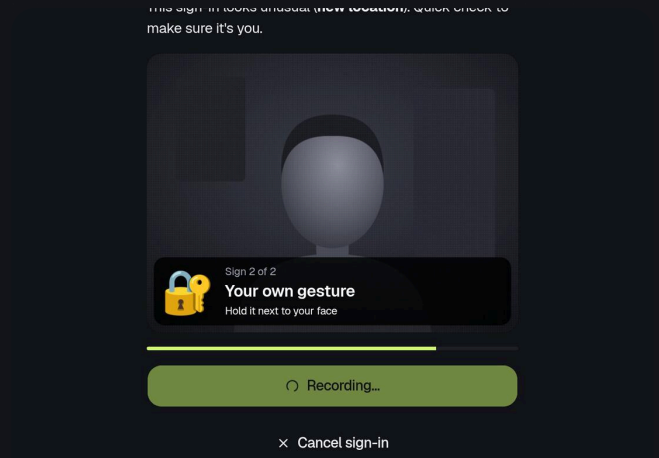
The admin feed logs `risk 40 · new_country · step_up_required`, then *Extra check passed*.

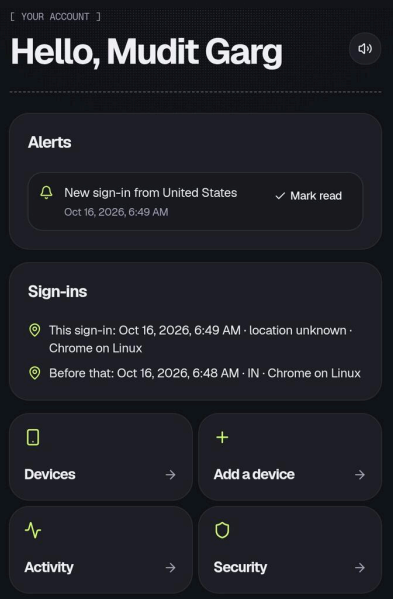
[04 · THE USP: FACE + GESTURE LIVENESS]

15 seconds that a photo can't pass

Two prompts, in random order: **your own gesture** plus **one random hand sign**. Each is held next to your face while bio checks it's the same live face throughout.







1 **Random canned sign**: here "Closed fist" / "Victory". 1 of 7 MediaPipe classes.

2 **Your own gesture**: the UI never knows what it looks like. Only bio has the template.

3 **Pass**: the session token rotates and the user gets an alert, "New sign-in from United States".

3 s get ready	**sign 1 · 4 s · hold ≥ 300 ms**	**sign 2 · 4 s · hand must change**	upload ≤ 80 JPEG frames → bio verdict

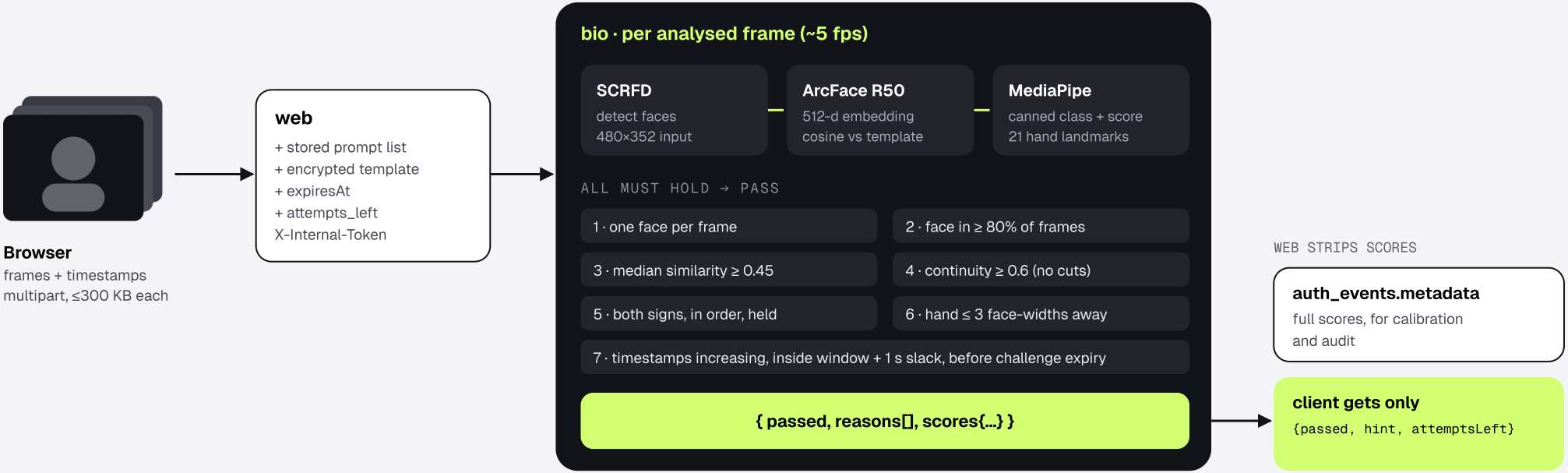
10 FPS · 640×480 · TIMESTAMPED

CHALLENGE TTL 120 S · 3 ATTEMPTS · 7 × 2 = 14 POSSIBLE ORDERS

[04 · LIVENESS ENGINE]

From raw frames to one bit

The decision happens in bio, from raw frames. The browser is never trusted. Scores go into the audit log, and the client only sees pass/fail plus a coarse hint.



No oracle

`face_mismatch`, `discontinuity`, `gesture_mismatch` and `template_invalid` all show the same hint, **"try again"**. An attacker can't tell which part failed.

Frames never touch disk

bio has its own in-memory multipart reader, because Starlette would spool parts over 1 MB to a temp file. Nothing is logged.

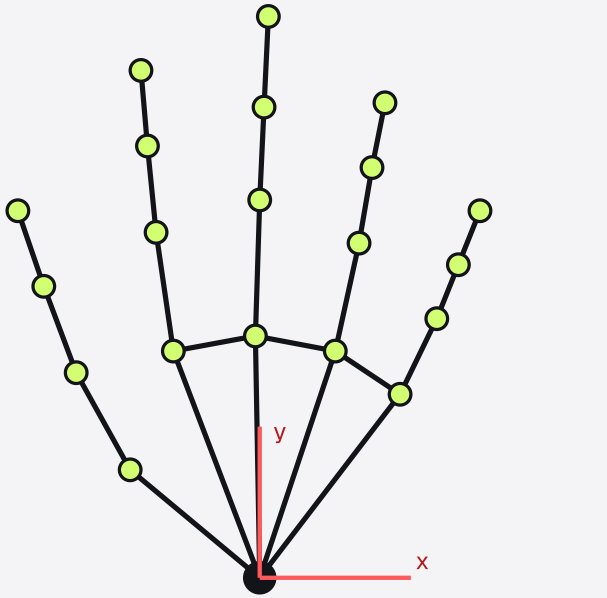
Fast on a laptop CPU

Analysing about every second frame cut verify time from ~8.5 s to ~4 s. Every analysed frame still gets all the checks.

[04 · CUSTOM GESTURE, NO TRAINING]

Your own hand shape as a second factor

MediaPipe gives 21 3-D landmarks per hand. We turn them into a 66-number shape vector and match it against the user's own samples with a nearest-neighbour search. No model training is needed.



wrist = origin · palm length = unit

SHAPE · 60 NUMBERS

The 20 non-wrist landmarks in a frame attached to the hand. Position, size and rotation don't change them. A mirrored hand does NOT match.

ORIENTATION · 6 NUMBERS

The hand frame's axes in camera space, lightly weighted, so 👍 and 👎 still count as different shapes.

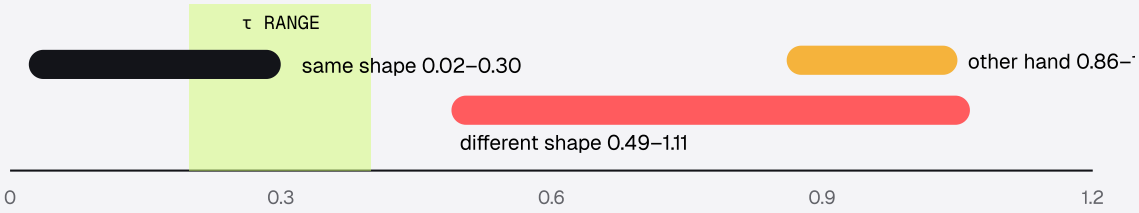
DISTANCE

Per-finger RMS. Error = (mean finger + worst finger) / 2, so a single bent finger isn't averaged away.

PER-USER THRESHOLD

`τ = clamp(p90 × 1.5, 0.2, 0.4)`, computed from cross-capture distances over 3 × 5 enrollment frames.

SYNTHETIC SEPARATION TEST (PALM LENGTHS)



MediaPipe sample hands under rotation, scale and JPEG noise. Live calibration with real users is still open (see p. 28).

[04 · SPOOF ATTEMPTS VS. CHECKS]

Layers that each attack has to beat

No single check is perfect, so every attack has to get past several at once. The passkey comes before all of them.

ATTACK	PASSKEY FIRST	IDENTITY	CONTINUITY	RANDOM ORDER	OWN GESTURE	HAND BY FACE	TIMING / TTL	ATTEMPT CAP	RESULT
Printed photo	•			•	•	•		•	STOPPED
Replay of an old check video	•			•			•	•	STOPPED
Spliced face clip + hand clip	•		•	•		•		•	STOPPED
Someone else's face	•	•						•	STOPPED
Guessing the custom gesture	•	•			•			•	STOPPED
One held shape for both steps				•	•		•		STOPPED
Relay face check via phishing page	•								401
Real-time deepfake, virtual camera	•	◖	◖	•	•	•	•	•	HARDER, NOT IMPOSSIBLE

14 sequences

Own gesture × 7 canned signs × 2 orders. A recording of an earlier check almost never has today's sign in today's order.

Hand must change

A step's hold starts only once the previous sign is gone. One held shape can't count twice.

Limits carry over

A fresh challenge doesn't reset attempts. Recovery is capped per request, counted from audit events.

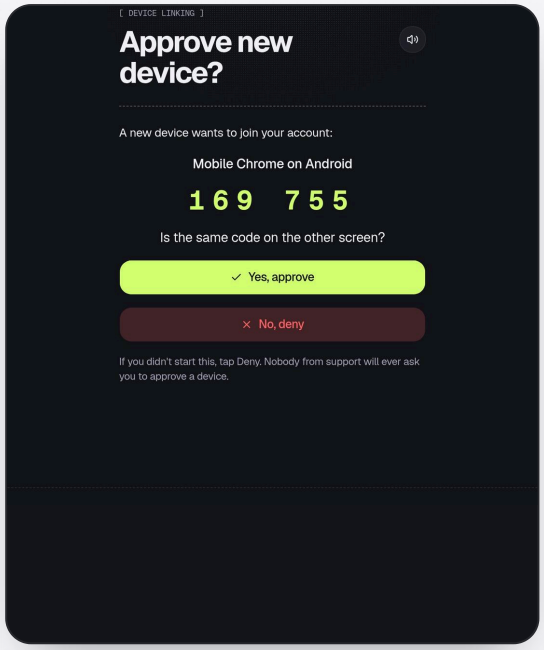
[05 · ADD A DEVICE]

Scan, match 6 digits, approve

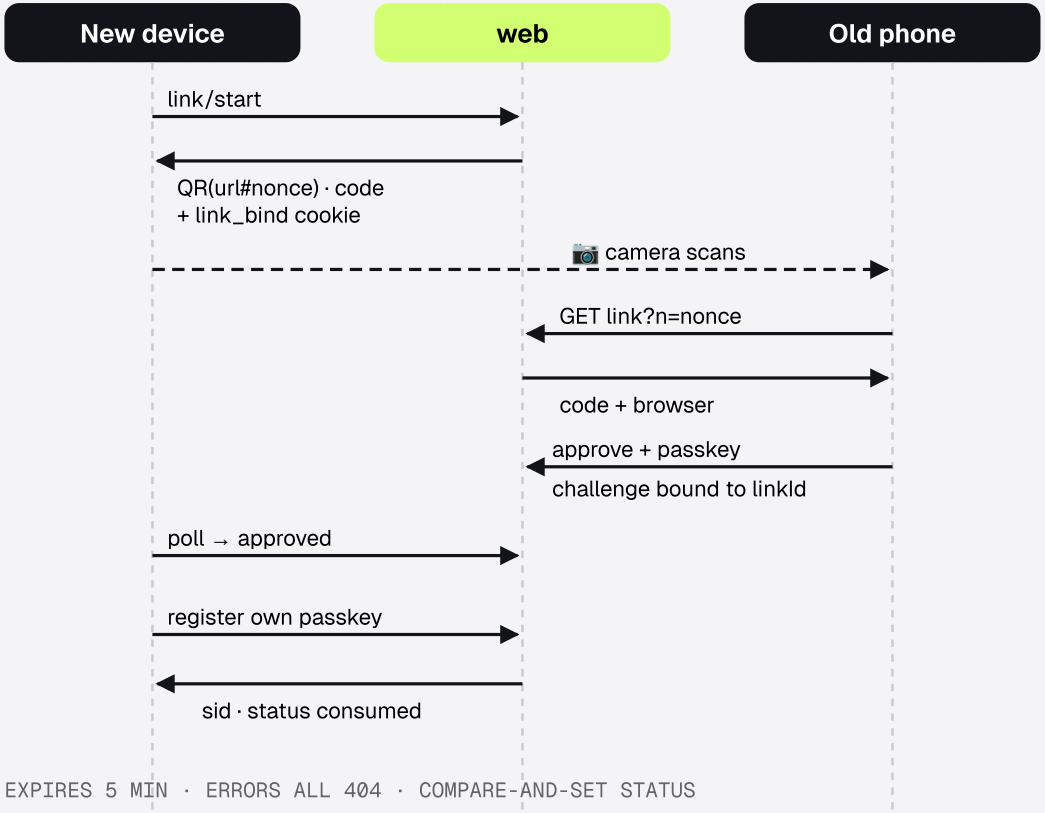
The new device shows a QR code and a short code. The signed-in phone scans it, confirms the digits match, and approves with its own passkey. Then the new device creates its own passkey.

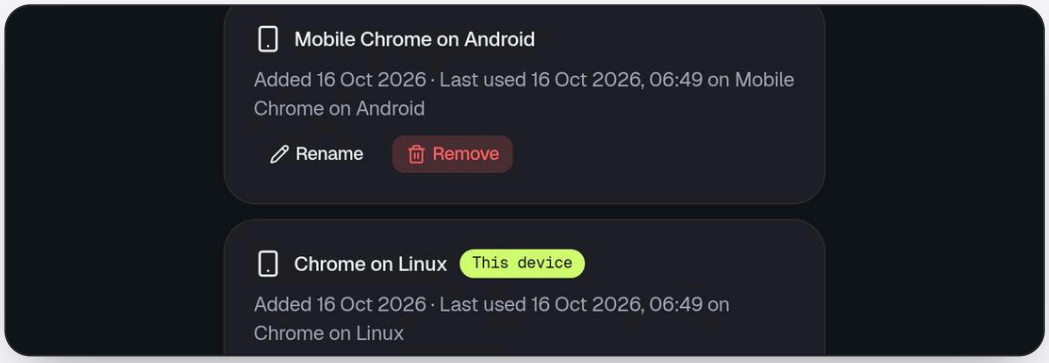


1 **New device** `/link`: QR + 6-digit code, polls every 2 s.



2 **Signed-in device**: "Same code on the other screen?" Approving requires a passkey assertion.



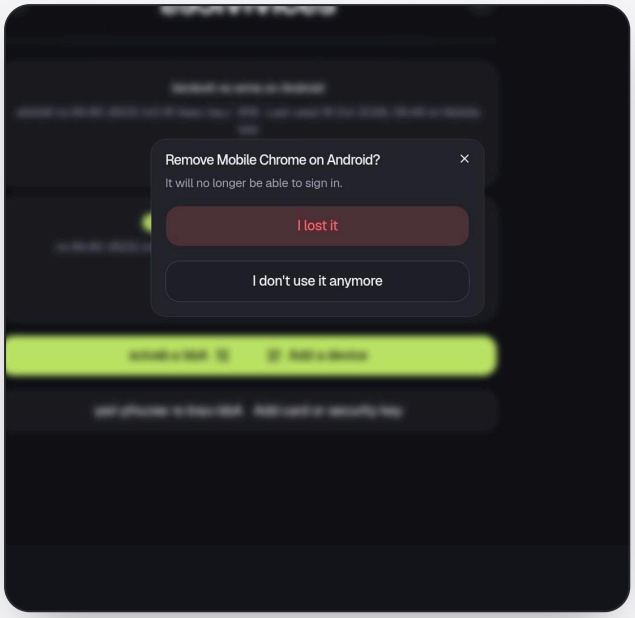


3 **/devices** now lists both: "Mobile Chrome on Android" + "This device".

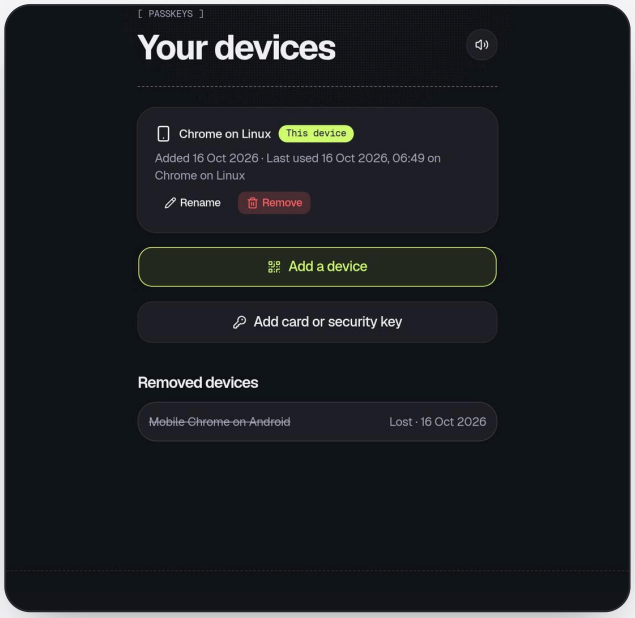
[05 · LOST A DEVICE]

Remove it, and its sessions die with it

Removing a device needs a passkey used in the last 5 minutes. The removed device is recorded with a reason, its sessions are revoked, and the user is notified.



1 **"I lost it"** or **"I don't use it anymore"**, both stored as the revoke reason.



2 Moved to **Removed devices** with a "Lost" badge. Its passkey can't sign in again.

GUARD RAILS

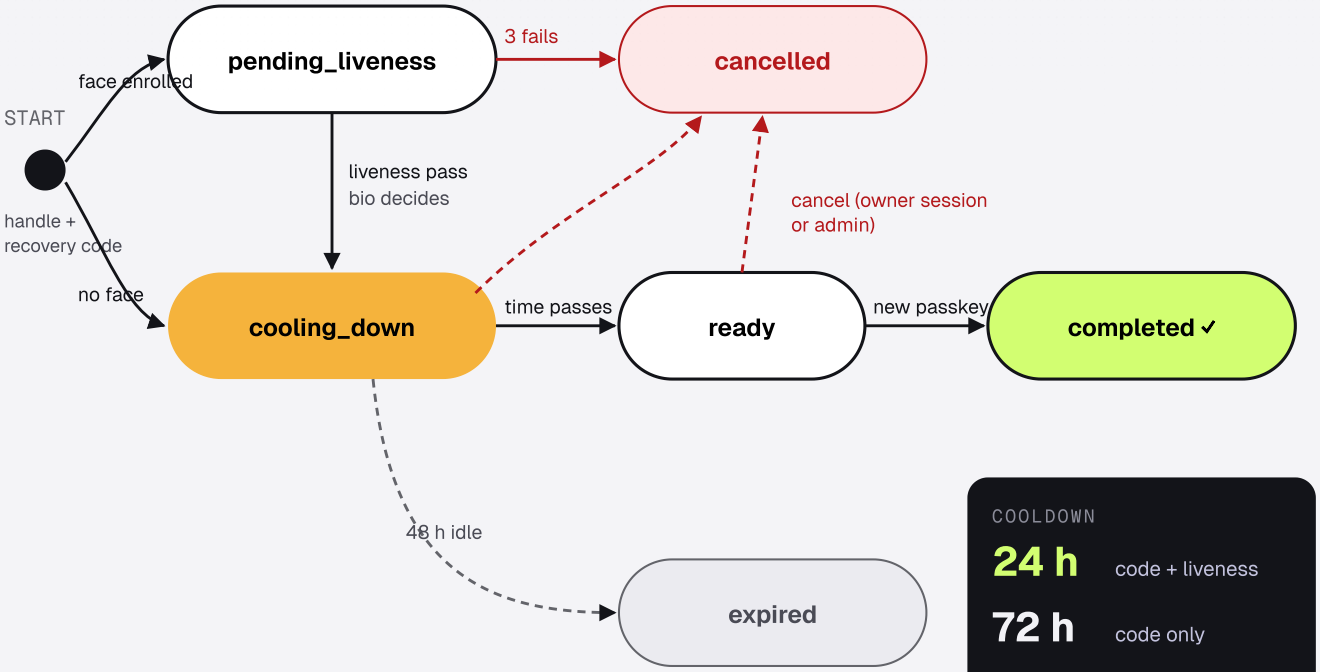
- **Reauth ≤ 5 min** to revoke, add a key, regenerate the code, change biometrics, or do any admin write
- **Last credential**: users get `LAST_CREDENTIAL` and are pointed to recovery. Admins may revoke it anyway
- **Removing the current device** signs it out right away
- **Security keys / NFC cards**: "Add card or security key" runs a normal registration labelled "Security key"
- **Cloned key**: a signCount regression auto-revokes it as `clone_detected`

Every row lands in `auth_events`: credential_revoked, reason: lost.

[06 · ACCOUNT RECOVERY]

Slow on purpose, loud on purpose

Recovery is the usual back door. Ours needs something you have (the code) and something you are (face + gesture), then waits, and tells every signed-in device so the real owner can cancel.



COOLDOWN

24 h code + liveness

72 h code only

demo override: 20–60 s

SIX LOCKS ON THE BACK DOOR

- **Recovery code**: 100 bits, argon2id, single use, replaced after recovery
- **Liveness**: same check as step-up, 3 failures cancel the request
- **Cooldown**: 24 h or 72 h before any new passkey
- **Alerts**: a banner + notification on every active session
- **Cancel**: owner (any session) or admin
- **Browser binding**: only the browser that started it can finish

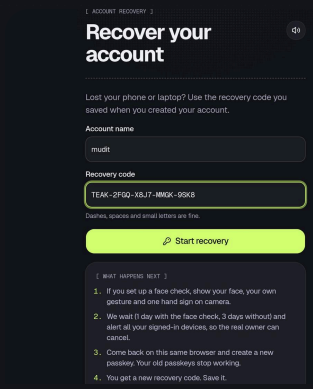
Completing recovery revokes **every** old credential and session in one DB transaction, gated on a conditional `ready → completed` update. A cancel racing a complete can't both win.

One pure function, `transition(request, event, now, cfg)`, unit-tested edge by edge. One open request per user.

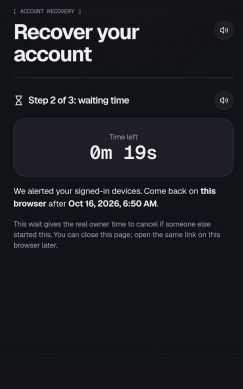
[06 · RECOVERY, END TO END]

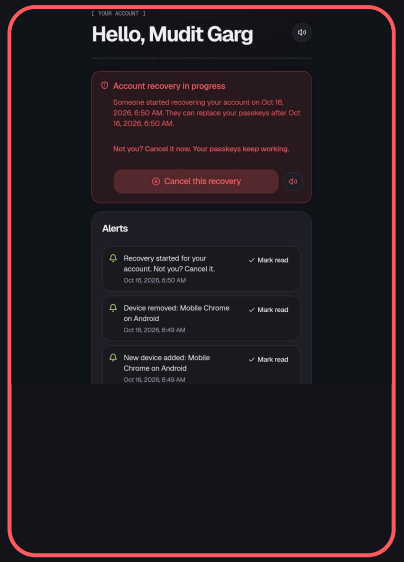
From "lost everything" to a fresh passkey

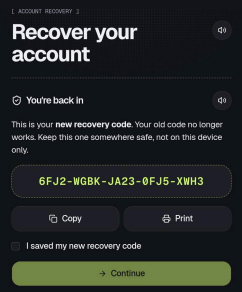
Captured live. Recovery runs on a new browser while the old laptop is still signed in, and the old laptop sees the alarm.











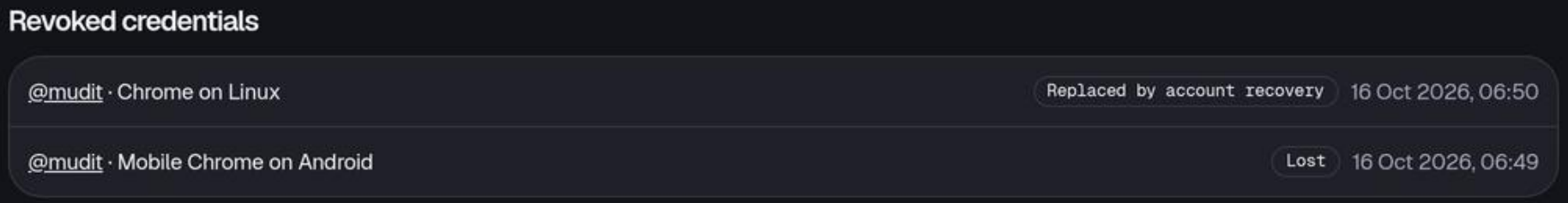
1 **Start**: handle + code. Bad input gives one generic error.

2 **Face check**: own gesture + a random sign.

3 **Wait**: countdown, "come back on this browser".

! **Old laptop**: red banner + "Cancel this recovery".

4 **Done**: new passkey + new code. Old devices are signed out.

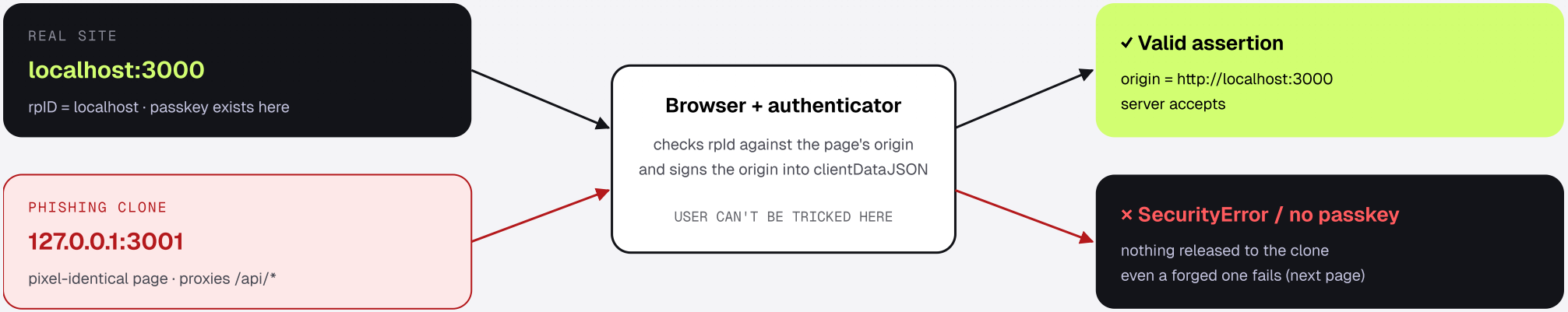


Admin → Revoked credentials: the old laptop is now marked **"Replaced by account recovery"** and the phone removed earlier is marked **"Lost"**.

[07 · PHISHING RESISTANCE]

A perfect clone still gets nothing

We built a look-alike sign-in page on a different origin (`127.0.0.1:3001`) with a proxy server, and tried every way to get a usable credential out of it.



#	ATTACK FROM THE CLONE	WHAT HAPPENED (LIVE LOG)	RESULT
1	**Ask for the real site's passkey** (`rpId: "localhost"`)	Browser: `SecurityError: This is an invalid domain.` Nothing released.	✓ BLOCKED
2	**Ask for a passkey for the clone's own rpId**	No passkey exists for that site. On a bare IP the browser refuses outright.	✓ BLOCKED
3	**Relay with a forged origin, using a stolen private key**	Valid signature, but `HTTP 400 ORIGIN_MISMATCH`, logged as `login_fail reason=origin_mismatch`	✓ BLOCKED
4	**Relay the face check** (`/api/stepup/challenge`)	`HTTP 401 UNAUTHENTICATED`. Liveness only exists after a real passkey login	✓ BLOCKED
5	**Contrast**: old-style password + SMS OTP form	Captures everything typed, in plain text. This is what passkeys remove.	CAPTURED

[07 · WORST CASE: THE PRIVATE KEY IS STOLEN]

Only the origin stopped it

The attacker exports the passkey's private key from a virtual authenticator, signs a real challenge with it, and forges the HTTP `Origin` header. The server still says no.

1 · Victim signs up on the real site

Chromium + CDP virtual authenticator

2 · Attacker exports the private key

WebAuthn.getCredentials, as if the authenticator itself leaked

3 · Sign a real challenge as the clone, forge Origin header

```
clientDataJSON.origin = http://127.0.0.1:3101
signature valid: yes → HTTP 400 ORIGIN_MISMATCH
```

4 · Control: same key, same trick, real origin

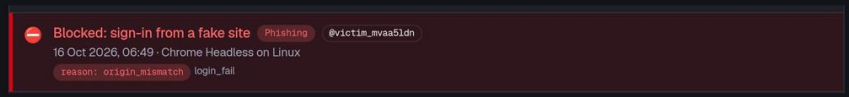
```
clientDataJSON.origin = http://localhost:3100
signature valid: yes → HTTP 200 {"status":"ok"}
```

Same key, same signature. The only difference is the signed origin.

```
[3] Phish server gets a real challenge, signs it
as the phishing origin, posts it with a forged
Origin header.
    challenge G3_qfKBJXgLnjhA9…
clientDataJSON.origin = http://127.0.0.1:3101
    signature valid for the stolen key: yes
    → HTTP 400 {"error":
{"code":"ORIGIN_MISMATCH","message":"This sign-in
came from a different site."}}
    ☑ REJECTED: ORIGIN_MISMATCH. The signed
origin gives the phishing page away. See /admin:
login_fail reason=origin_mismatch.
[4] Control: same stolen key, same server trick,
but clientDataJSON.origin = the real origin.
    challenge KxITzg9CW57z8C_C…
clientDataJSON.origin = http://localhost:3100
    signature valid for the stolen key: yes
    → HTTP 200 {"status":"ok"}
    (accepted: the key and signature were fine,
ONLY the origin stopped step 3. A phishing page
can't produce this.)
  attack 3 rejected as expected

[5] Contrast: a fake password + SMS OTP form.
Type anything and press Sign in.
```

The clone's "attacker console" log during the run.

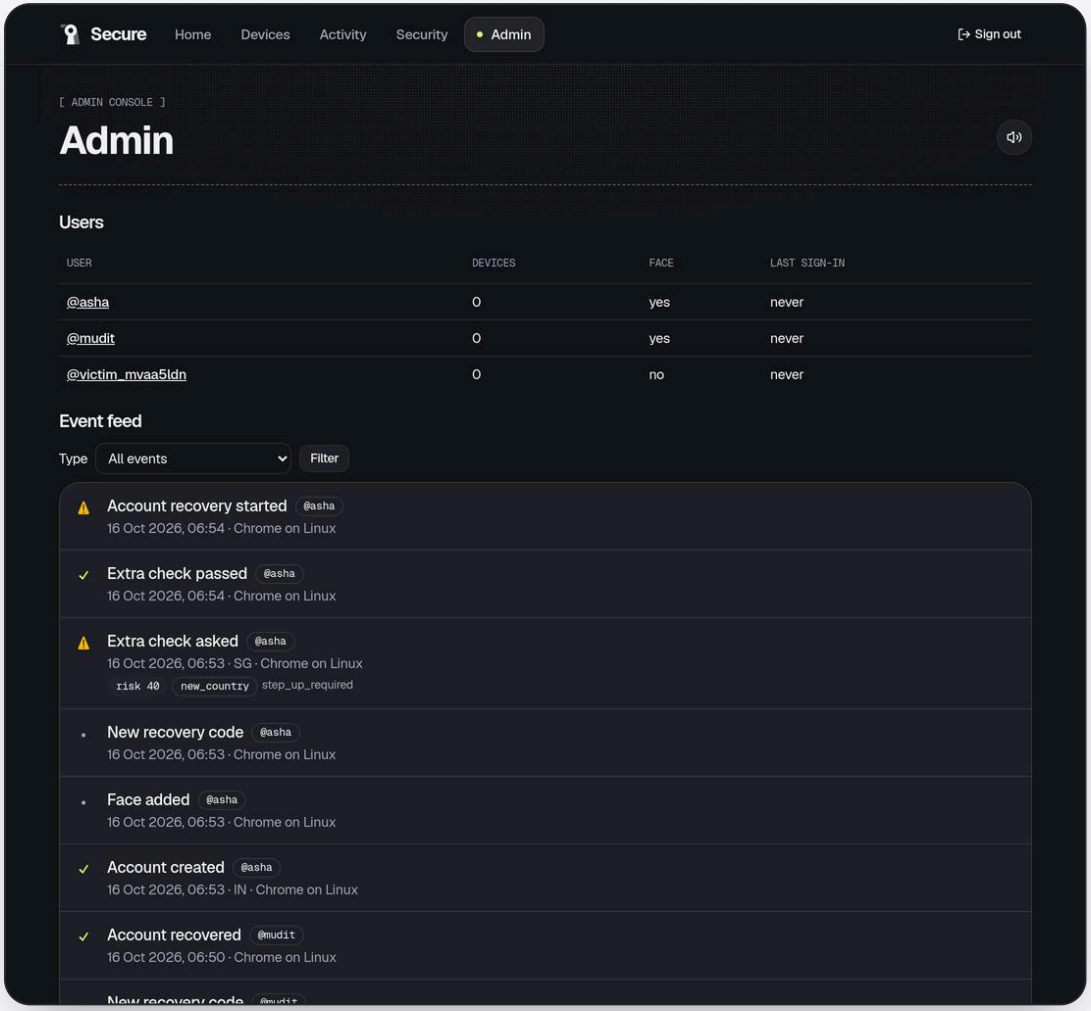


…and the real site's admin feed: **"Blocked: sign-in from a fake site"**, tagged Phishing.

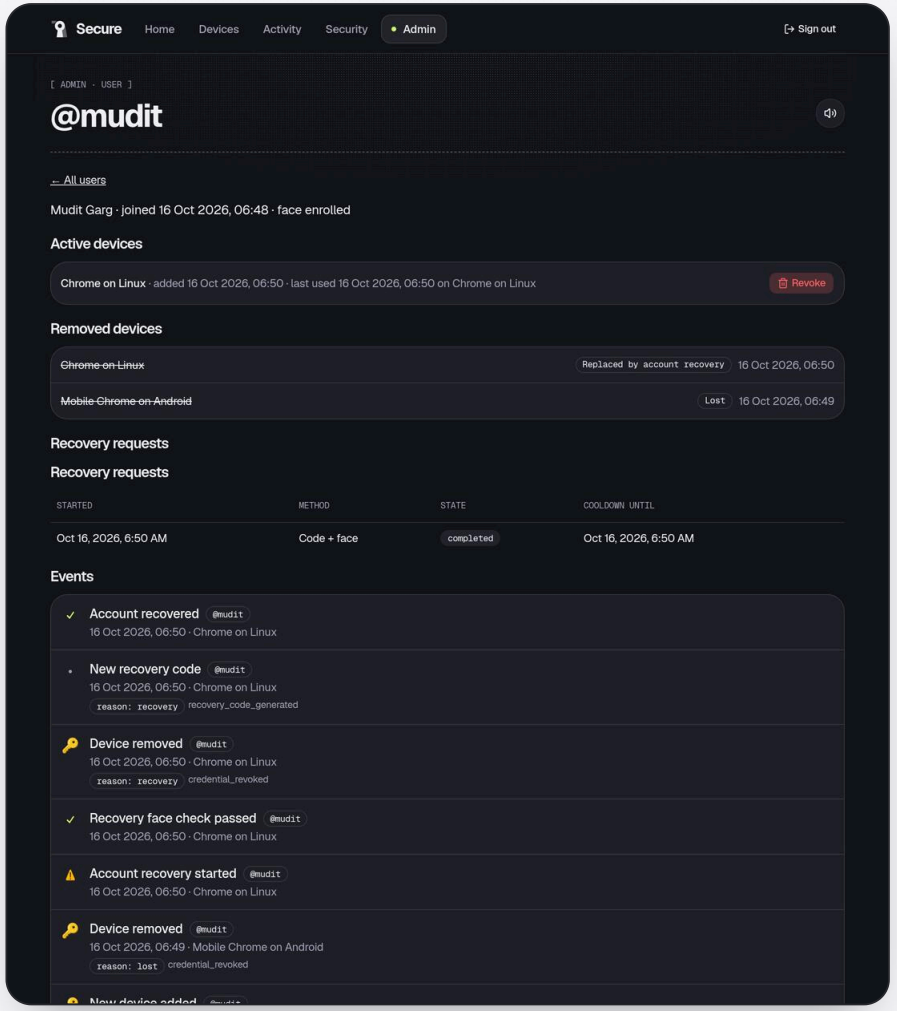
[08 · ADMIN VIEW]

Devices, history, revocations: one console

The pages are server-rendered straight from SQLite. Admin writes (revoke a credential, cancel a recovery) need a fresh passkey and log an `admin_action` event.



/admin: users, a filterable event feed (risk signals as chips), revoked credentials.

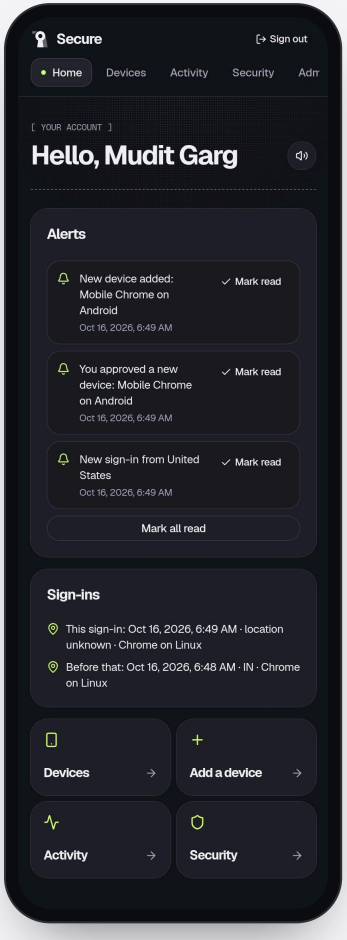


/admin/users/[id]: active + removed devices with reasons, recovery requests, the user's full timeline.

[08 · INCLUSIVITY]

Works on a basic phone, without reading much

The users the problem statement worries about: basic phones, shared devices, limited literacy. Every screen has one main action, a big icon + text button, and a 🔊 button that reads it aloud.



Phone layout at 390 px: big tiles, read-aloud on the heading.

📱 Phone as a key

No modern laptop? On a shared or library PC, "Use phone or security key" shows a QR code, and the user's phone signs in over hybrid transport (Bluetooth proximity).

💳 NFC card / security key

For users without a smartphone: a cheap FIDO2 card or key works for login and as an extra device. No app needed.

🔊 Voice on every screen

Browser `speechSynthesis` reads the heading and instructions. Liveness prompts are spoken too: "Victory sign. Hold it next to your face."

👆 No typing to sign in

Passkeys are offered in autofill, or with one big button. Recovery codes accept dashes, spaces and lowercase.

✋ Pick your own gesture

Any hand shape you can repeat. Nothing to remember, read or spell. Prompts use an emoji + one or two words.

🧭 Plain language

"I lost it", "Is the same code on the other screen?", "Not you? Cancel it now." No jargon or error codes in the UI.

360 px Every flow was checked at 360 px width. Buttons are full-width with icon + text, and the QR code + 6-digit code are readable across a room.

[09 · THREAT MODEL]

What we stop, what we slow, what we don't

The deliverable short threat model. It is also published in the app at `/threat-model`.

COVERED

Phishing / look-alike site	origin + rpID binding, server origin check
Password reuse, stuffing, breach dumps	no passwords. The server holds public keys only
SIM swap, OTP interception	no OTPs anywhere
Database leak	public keys, argon2id hashes, AES-GCM blobs
Photo / replayed video at step-up	random sign + own gesture, bound to the face, tight window
Spliced face + hand video	embedding continuity + hand/face co-presence
Stolen unlocked device	passkey needs device biometric/PIN. Revoke remotely
Cloned authenticator	signCount regression → auto-revoke
Session theft	HttpOnly cookie, hashed in DB, expiry, rotation
Takeover via recovery	code + liveness + cooldown + alerts + cancel + binding
Approval replay	approval challenge bound to the link ID

PARTIALLY COVERED

Real-time deepfake via virtual camera	harder, not impossible. Passkey still needed first
Victim talked into reading code + doing liveness	cooldown + alerts let the real owner cancel
Lost every device	recovery works, but no device gets the alert
Basic-phone users	NFC card / key / phone-as-key. No IVR / USSD

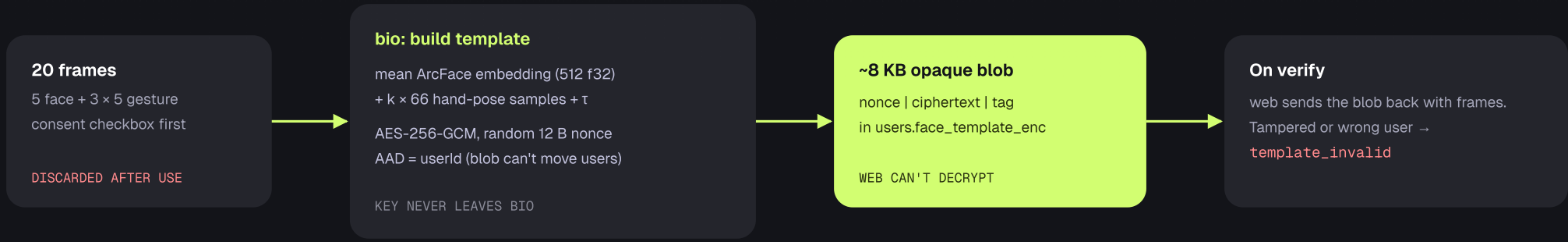
NOT COVERED

- Malware controlling the user's device or browser
- Coercion
- Compromise of passkey sync providers
- Attacker with both the DB *and* `BIO_TEMPLATE_KEY`
- Rate limiting / DoS (only per-challenge limits)
- Certified liveness (ISO/IEC 30107-3), anti-spoof model, commercial face weights

[09 · PRIVACY & CRYPTOGRAPHY]

Your face never becomes a file

Biometrics are processed in memory, stored only as an encrypted template that the web server can't read, and can be deleted by the user at any time.



NEVER STORED OR LOGGED ANYWHERE

- raw frames
- recovery codes
- session tokens
- match scores → client
- passwords (none exist)

User control

Enroll, re-enroll or delete face + gesture on `/security`. After deleting, risky logins are blocked and recovery uses the longer 72 h path.

Minimal at rest

Session = sha256 of a 32-byte cookie. Recovery code = argon2id. WebAuthn user ID = 32 random bytes, never the handle.

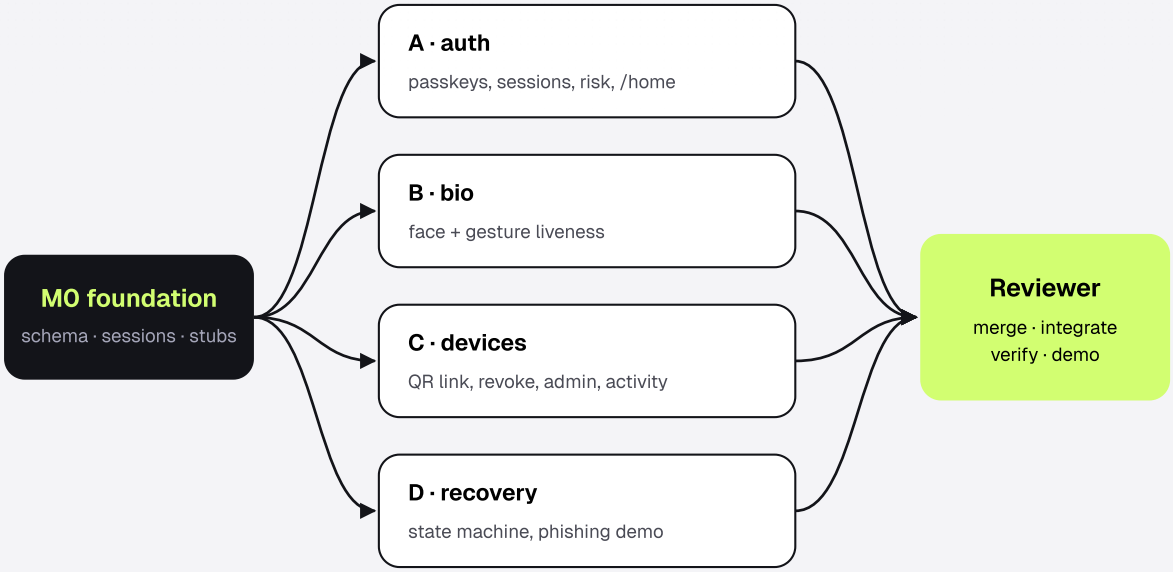
Honest about limits

InsightFace weights are non-commercial and there's no certified anti-spoof model. Both are listed as gaps, not hidden.

[09 · ENGINEERING & QUALITY]

Built in parallel, merged clean

A shared foundation landed first. Then four agents each built one area in its own git worktree, with path ownership enforced by a script, and a reviewer merged and verified the result.



`scripts/owners.txt` + `check-scope.sh` before every commit

120

WEB TESTS · VITEST
9 FILES · ALL PASS

121

BIO TESTS · PYTEST
5 FILES · ALL PASS

WHAT THE TESTS PIN DOWN

- Risk scoring, every signal and threshold
- Recovery state machine, edge by edge
- Bio decision rules: order, holds, splice, timing
- Hand-pose features: invariance, mirror, thumbs up vs down
- Template crypto: tamper, wrong user, wrong key
- Bio API: token, multipart, oversize parts

INVARIANTS ENFORCED IN CODE

zod on every route · `requireSession()` everywhere · Origin header check on mutations · all tunables in `config.ts` / `config.py` · append-only `auth_events`

[10 · LIMITS, ROADMAP, DEMO]

What's next, and the 6-minute demo

This is a hackathon build. It is honest about the gap to production and has a rehearsed path for showing everything live.

BEFORE REAL DEPLOYMENT

Calibrate thresholds with real people
Face 0.45 and pose τ are starting values validated on synthetic data. They need a live calibration session.

Licensed face model + passive anti-spoof
Swap the non-commercial InsightFace weights and add MiniFASNet on face crops.

Rate limiting, real domain + HTTPS
rpID must match the domain. Add per-IP and per-user limits beyond the per-challenge caps.

Out-of-band recovery alerts
Email alerts so users who lost every device still hear about a recovery.

STRETCH IDEAS

MOVING GESTURES (DTW) · LIVE IN-BROWSER PROMPT TICKING · HINDI UI · PLAYWRIGHT E2E · IVR / USSD

DEMO SCRIPT

1. **Sign up**: passkey, face + own gesture, save the recovery code. Sign in normally.
2. **Simulate a new country** → step-up. Fail with a photo, pass live.
3. **Add a laptop by QR**, then remove the old phone as lost.
4. **Phishing clone**: every attack fails, including the stolen-key relay.
5. **Recovery**: code + liveness, the banner on the other session, complete after a short cooldown.
6. **Admin**: devices, login history, revoked credentials, blocked phishing attempts.

Run it: `./dev.sh --phish` starts bio :8000, web :3000, landing :4321 and the clone :3001.



Nothing to phish.

Passkeys log you in. Your face and hand prove it's really you when it matters.

SECURE · CC6-IC04 · OCTOBER 2026